



INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA (ISEL)

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA (DEI)

MEIM

MESTRADO EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA
ENGENHARIA DE SOFTWARE

Relatório Projeto Final

Miguel Alcobia (50746)

Docente

Professor Luís Morgado

Janeiro, 2026

Índice

Índice	i
Lista de Tabelas	iii
Lista de Figuras	v
1 Introdução	1
2 Modelação de Software	3
2.1 Ferramenta utilizada	3
2.2 Modelo Domínio	4
3 Análise de requisitos	7
3.1 Análise	7
3.2 Especificação de Requisitos	8
4 Projeto de Arquitetura da Solução	11
4.1 Modelo de Dinâmica	11
4.2 Modelo de Interação	13
4.3 Arquitetura de Software	15
4.3.1 Métricas	15
4.4 Arquitetura Detalhada	17
5 Implementação do Protótipo da Solução	19
5.1 Acesso aos Dados	19
5.2 Domínio	20
5.3 Apresentação	20
5.4 Testes	21

6	Revisão do Projeto Realizado	23
7	Conclusão	27
	Bibliografia	29

Lista de Tabelas

3.1	Exemplo de como os requisitos foram apresentados na Especificação Suplementar	8
-----	---	---

Lista de Figuras

2.1	Modelo de Domínio	5
3.1	Diagrama do Contexto de Utilização usado na Visão	8
3.2	Diagrama com os casos de utilização usado na Especificação de requisitos	10
4.1	Modelo de dinâmica da Criação da Reserva	12
4.2	Modelo de dinâmica do Check-in	13
4.3	Diagrama de Criar Reserva	14
4.4	Diagrama que mostra o fluxo de funcionamento do padrão MVC [GeeksforGeeks, 2025]	16
4.5	Arquitetura de Teste	18
5.1	Estrutura da implementação	22

Capítulo 1

Introdução

Este documento apresenta todo o processo e evolução do projeto feito no âmbito da unidade curricular de Engenharia de Software do Mestrado de Engenharia Informática e Multimédia. O tema escolhido para o trabalho foi o de gestão de hotéis e, portanto, o trabalho durante o semestre seria criar um sistema para tal.

Sendo o projeto um método de avaliação, o objetivo principal seria aplicar os conceitos lecionados. Contudo, ao longo do desenvolvimento do trabalho, foram encontradas algumas dificuldades.

A primeira foi lidar com a complexidade que é devida a um sistema destes. Mesmo que o escopo do sistema seja pequeno, tenha poucos casos de utilização e poucos atores, ainda apresenta uma complexidade desafiadora. Em segundo lugar, existiu um grande desafio em elaborar os diagramas de interação.

Por último, encontrar a melhor forma de representar determinados conceitos do sistema na linguagem UML. Embora a linguagem seja bastante flexível e completa, por vezes, era difícil encontrar a melhor forma e usá-la para representar as ideias a implementar no projeto.

Contudo, estas dificuldades foram ultrapassadas, na sua maioria, ao consultar e reler melhor os materiais de apoio da unidade [Morgado, 2025] curricular e ao esclarecer as dúvidas com o professor.

Capítulo 2

Modelação de Software

A modelação de software é um dos meios principais para lidar com a complexidade do software. Devido a esta natureza, foram aplicados os conceitos principais que definem esta técnica: **Abstração, Representação e Modelo**.

A Abstração ajuda a lidar com a complexidade ao realçar apenas as características relevantes e comuns a diferentes partes.

A Representação expressa o conhecimento sobre um determinado domínio através de conceitos que o definem. É a base para os conceitos que fundamentam os diagramas de fluxo e a modelação de dados.

Surgiram linguagens para representar sistemas de forma abstrata sob a forma de modelos como a linguagem UML (*Unified Modelling Language*). Esta linguagem possibilita realizar vários diagramas sob perspectivas diferentes (classes, transição de estado, interação, casos de utilização etc.).

2.1 Ferramenta utilizada

A ferramenta utilizada foi o MagicDraw, com uma licença académica fornecida pelo ISEL. Optou-se por esta escolha, pois existia o benefício da licença e já tinha alguma familiaridade com o programa, por mexer com ele durante a licenciatura.

Ao abrir o programa percebe-se que ele é antigo (2006) e causa alguma estranheza nas primeiras interações que se tem com ele. Contudo, também é perceptível a quantidade de funções e o quão completo é o programa. É difícil, no começo, navegar pelas diferentes funções e menus que o programa

disponibiliza, mas não sei se é devido aos anos que programa ou se atribua esse facto à quantidade de funções que oferece, porque acredito que não é fácil, sobretudo depois deste projeto, organizar todas as funções de uma forma perfeita. Acredito que os programadores na altura fizeram o melhor que conseguiram.

O ponto negativo que deixo ao MagicDraw é de apagar algumas das mensagens criadas nos diagramas de interação. Desconheço a razão do problema e o facto de parecer não haver um padrão para a eliminação delas dificulta encontrar a raiz do problema. Contudo, como exportava o trabalho realizado para PDF frequentemente, não se perdeu nenhum detalhe.

2.2 Modelo Domínio

O Modelo de Domínio é um modelo conceptual que representa os conceitos do domínio do problema e as respetivas relações [Morgado, 2025]

Este tipo de modelo não inclui pormenores que dizem respeito à implementação. Foca-se apenas nos conceitos presentes no domínio do problema e nas suas relações conceptuais. Primeiro identificaram-se os conceitos que fundamentavam o problema, como Reserva, Quarto, Hóspede e Funcionário, por exemplo. Esta seleção foi feita tendo por base que quando um elemento fosse representado por um nome, seria representado por uma classe ou atributo e, no caso de ser um verbo, indicaria uma operação ou relação. Depois filtraram-se os nomes obtidos de forma a eliminar ambiguidade e redundância.

Em seguida, elaborou-se, iterativamente, o diagrama apresentado na Figura 2.1. Prosseguiu-se para a definição das relações entre classes e dos seus atributos.

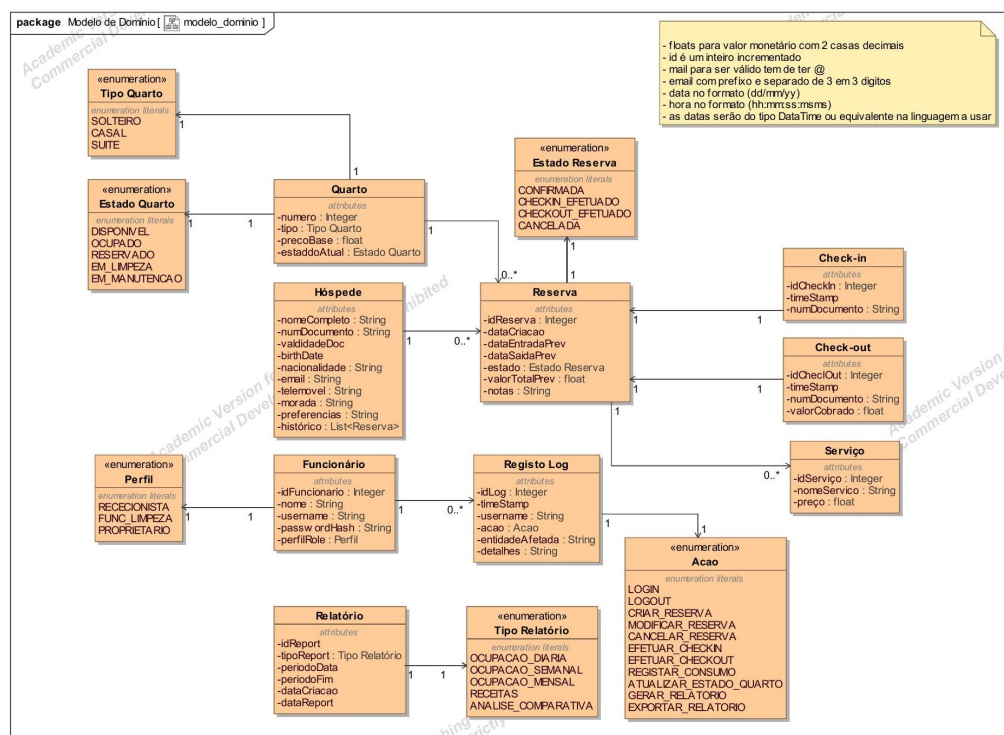


Figura 2.1: Modelo de Domínio

Capítulo 3

Análise de requisitos

3.1 Análise

Por análise de requisitos entende-se o estudo do contexto sobre o qual se vai trabalhar e de eventuais sistemas já existentes para perceber melhor a origem do problema para ser possível elaborar uma possível solução e definir os requisitos necessários para tal.

Um requisito é uma característica ou uma capacidade do sistema que é necessária para que este consiga realizar o objetivo desejado.

Primeiro começou-se por analisar o problema em questão: a dependência de processos manuais e as várias folhas de cálculo (Excel) envolvidas, no contexto da gestão de um hotel.

Tendo conhecimento do problema, partiu-se para a elaboração da visão de solução (ver `es_50746_visao`), que resume os resultados desta fase. Descreve o problema de forma geral, as partes envolvidas e interessadas (funcionários, proprietários do hotel e hóspedes). A partir deste documento, tem-se uma boa base para começar a especificar os requisitos propriamente ditos.

Embora existam técnicas como entrevista e questionários para ajudar a identificar os requisitos, devido à natureza do projeto estar sob um contexto académico, optou-se pela simulação de papéis de utilização e casos de utilização.

Na simulação de papéis, colocamo-nos no lugar de quem vai usar o sistema e tentamos perceber o que seria bem-vindo num novo sistema.

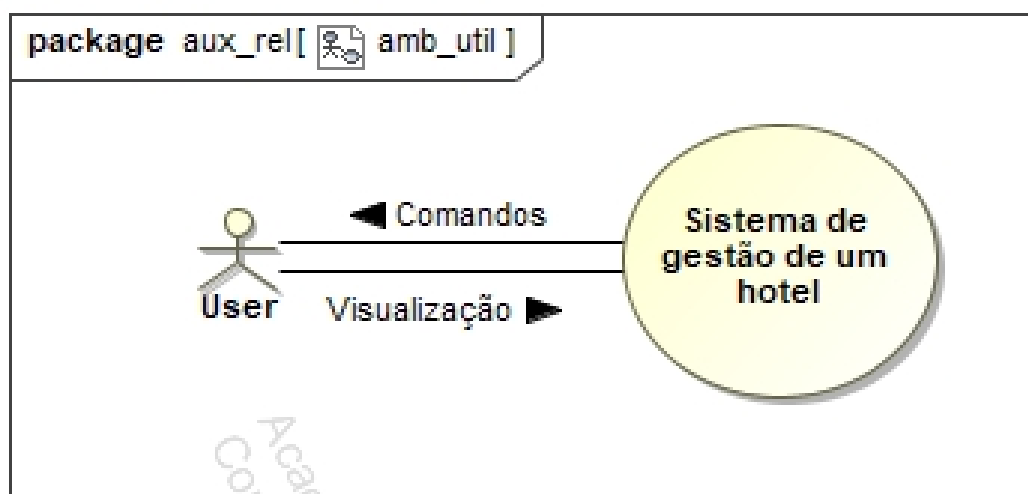


Figura 3.1: Diagrama do Contexto de Utilização usado na Visão

3.2 Especificação de Requisitos

Após a elaboração da visão, passou-se para a especificação de requisitos (ver `es_50746_requisitos`). Esta fase do desenvolvimento tem como objetivo estabelecer um ponto de acordo entre o cliente e a equipa de desenvolvimento sobre as características do sistema, descrevendo o que o produto final deverá fazer.

Para um melhor entendimento do projeto, optou-se por adotar uma linguagem natural estruturada, simples e objetiva, para apresentar estes requisitos. Desta forma, tanto a equipa de desenvolvimento quanto o cliente conseguem compreender as características do sistema, apesar da diferença de conhecimento técnico entre ambos.

Ref.	Descrição	Categoria
R10	Todas as passwords devem ter complexidade mínima: 8 caracteres, com letras maiúsculas, minúsculas e números.	Obrigatório

Tabela 3.1: Exemplo de como os requisitos foram apresentados na Especificação Suplementar

Para descrever cada caso de utilização em detalhe, usaram-se conceitos-

base como **Ator**, **Caso de Utilização** e **Sistema**.

Um Ator representa uma entidade que interage com o sistema, desempenhando um papel específico. No contexto deste projeto, todos os atores correspondem às partes interessadas identificadas na Visão (menos o hóspede, que não terá um contacto direto com o sistema).

Um Caso de Utilização representa a sequência de ações que os utilizadores podem realizar no sistema para obter um determinado resultado.

Para cada caso, identificou-se o **nome**, os **atores**, um cenário principal e outros alternativos, quando necessário.

Um cenário principal descreve o fluxo normal de eventos, de forma otimista, como se a interação entre os atores e o sistema não desse nenhum erro. Por outro lado, um cenário alternativo descreve as situações nos quais esses erros surgem.

Estes cenários foram descritos passo a passo e enumerados para que o entendimento sobre eles fosse facilitado.

Para finalizar a especificação de requisitos, realizou-se a especificação suplementar, que apresenta os requisitos não funcionais. Estes requisitos foram apresentados como mostrado na tabela 3.1 mostrada acima.

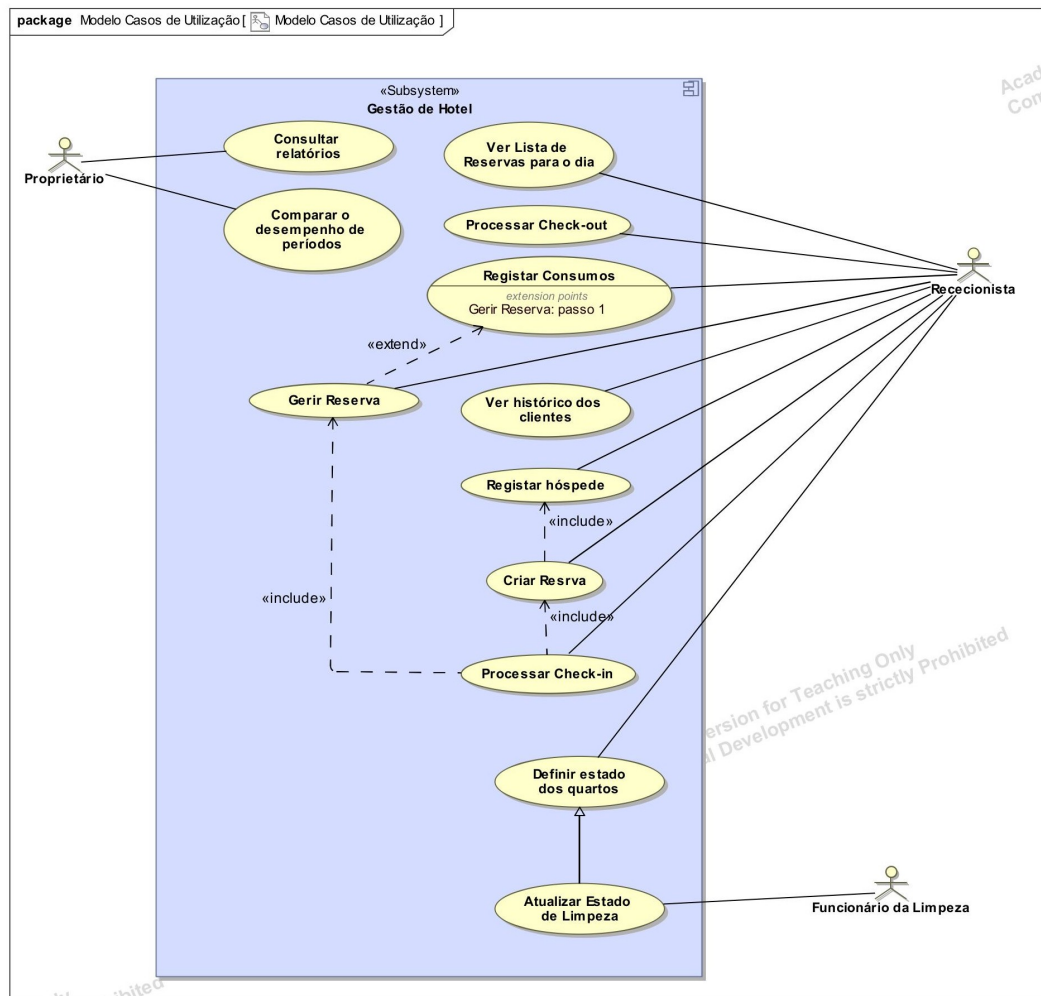


Figura 3.2: Diagrama com os casos de utilização usado na Especificação de requisitos

Capítulo 4

Projeto de Arquitetura da Solução

4.1 Modelo de Dinâmica

A dinâmica de um sistema define o comportamento que o sistema pode vir a assumir com a configuração das suas partes (estados), que podem mudar e evoluir ao longo do tempo.

Para representar esta dinâmica, recorreu-se a modelos de dinâmica para demonstrar a evolução no tempo e os estados que o sistema pode assumir e a forma como eles evoluem.

Os principais conceitos usados foram os de **Estado**, **Transição**, **Evento** e **Máquina de estados**.

Um Estado representa a configuração que o sistema, ou uma parte dele, pode assumir. Por sua vez, uma Transição é a representação da alteração do estado devido a um ou mais eventos. Um Evento é uma ocorrência no tempo e no espaço que determina uma evolução no estado.

As Transições podem ser definidas através de um evento, de uma Guarda e uma Ação. Uma guarda apresenta a condição que inibe ou permite as transições ou ações [Morgado, 2025], enquanto uma ação representa um comportamento que está associado a uma transição e a um estado.

Uma Máquina de Estados representa os estados e as transições válidas do sistema.

Para eliminar um problema de repetição na definição das transições, optou-se por recorrer a Ações de Entrada e de Saída. Estas ações definem no

próprio estado as ações que se executam à entrada e à saída com um **entry** e um **exit**.

Esta alteração também ajudou a reduzir a complexidade dos diagramas (ver `es_50746_arquitetura` - Modelos de Dinâmica).

Também foi usado o conceito de sub-estado, que é um estado composto, definido por uma máquina de estados própria (por exemplo, no diagrama da Figura 4.2).

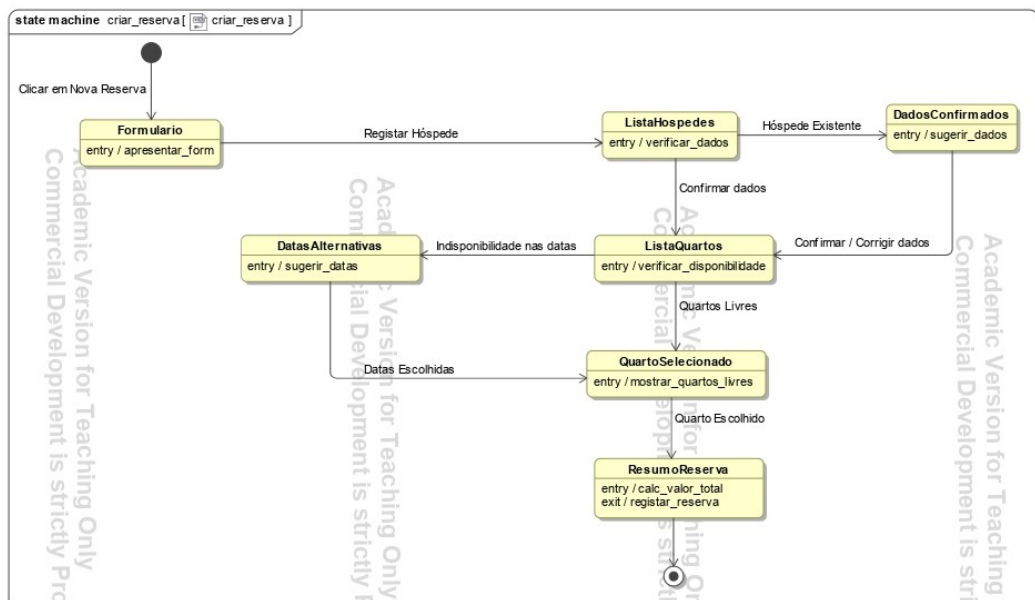


Figura 4.1: Modelo de dinâmica da Criação da Reserva

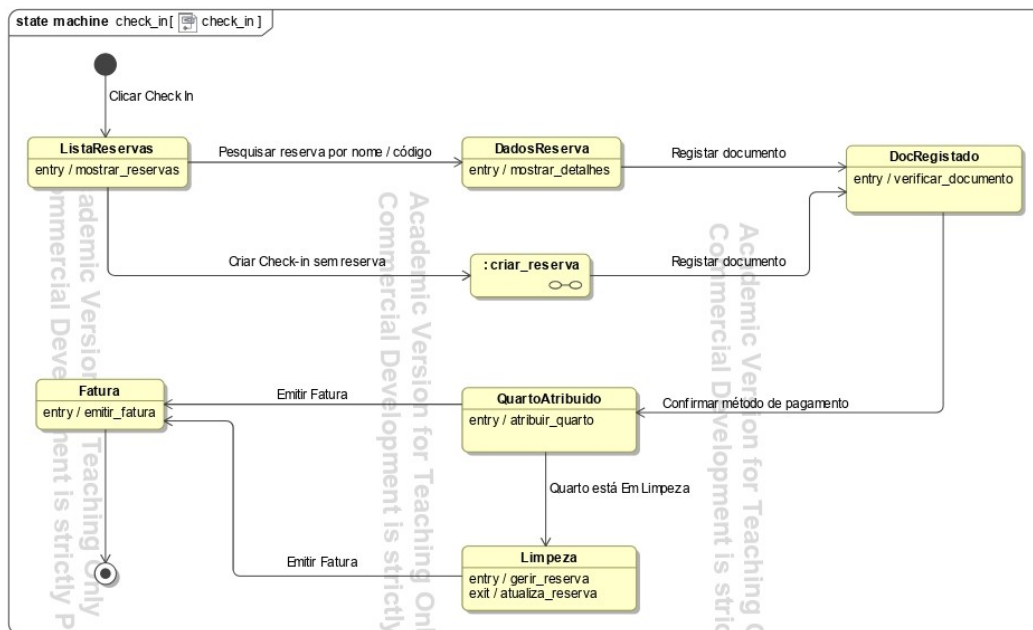


Figura 4.2: Modelo de dinâmica do Check-in

4.2 Modelo de Interação

Um modelo de interação mostra como as partes do sistema interagem entre si e com o exterior para produzir a uma respetiva funcionalidade. [Morgado, 2025]

Este tipo de modelos ajuda a ter uma melhor visão do sistema como um todo durante a concepção e também é uma boa ferramenta para verificar o bom funcionamento do sistema durante a realização dos testes.

Na linguagem UML estes modelos são representados com modelos de interação. Existem vários tipos destes modelos, mas neste projeto foram usados apenas diagramas de sequência, que têm um foco maior em representar a interação entre as diferentes partes do sistema ao longo do tempo.

Estes diagramas são compostos principalmente por:

- **Parte**- Representa uma instância de uma objeto que participa da interação.
- *Lifeline* - Representa a existência de uma parte ao longo ao tempo e indica a duração da existência desse objeto.

- Mensagem - Representa a comunicação entre as partes. (Podem ser chamadas de métodos, respostas ou sinais diversos)
- Foco de Ativação - Representa o período no qual a parte está ativa,
- Operador - Delimita uma interação com uma semântica específica.

As partes, nesta fase do projeto, deveriam já ter um nome que apareceria na arquitetura lógica. Contudo, devido a estes diagramas terem sido feitos antes dessa parte da arquitetura, existe uma diferença no nome das partes, mas o significado é semelhante e facilmente se percebe a renomeação feita.

Mesmo assim, os diagramas da arquitetura lógica têm uma pequena nota, própria do UML a avisar quando existe uma ocorrência deste tipo.

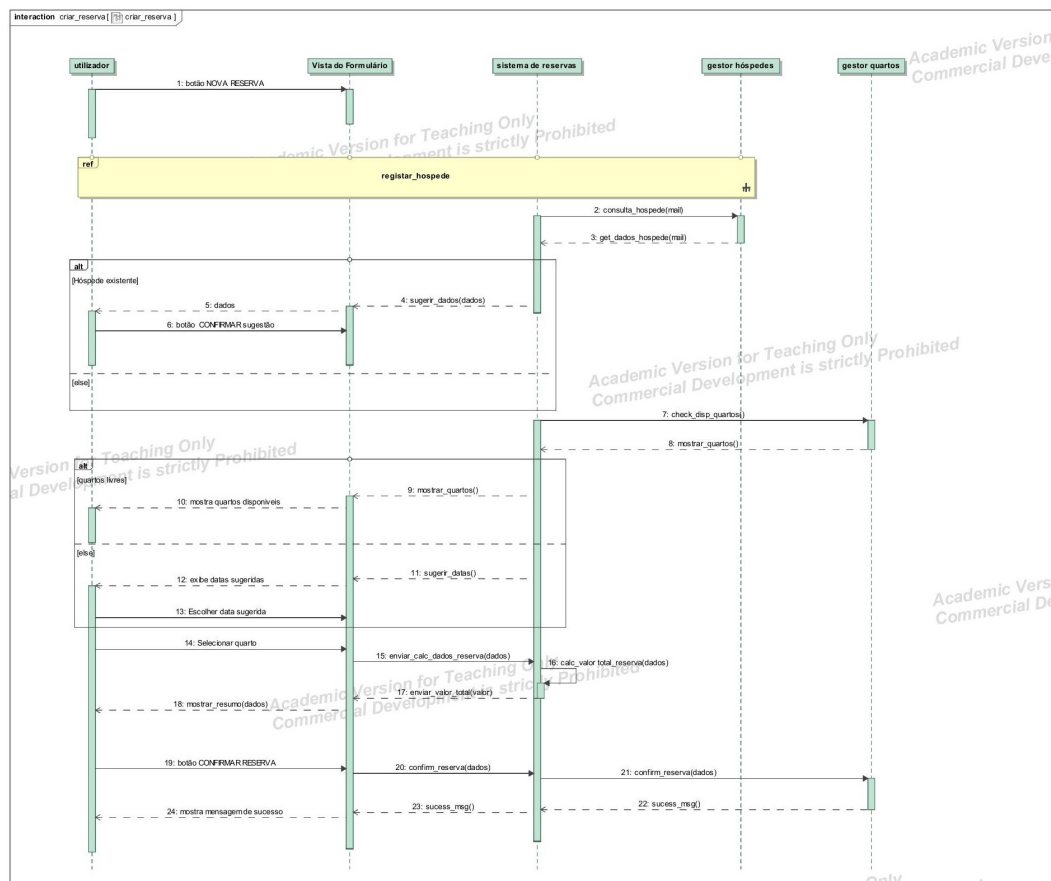


Figura 4.3: Diagrama de Criar Reserva

4.3 Arquitetura de Software

A arquitetura de Software define a organização do sistema, com foco nos componentes, nas relações entre eles e o ambiente.

Podemos dividir esta arquitetura em três níveis de abstração:

- Conceptual - Representação de alto nível dos conceitos do sistema (ex: Modelo do Domínio)
- Lógica - Mais concreta na organização que a anterior, mas ainda é independente da tecnologia
- Física - É a representação mais concreta e inclui pormenores de implementação dependentes da tecnologia usada.

4.3.1 Métricas

Para averiguar a qualidade de uma arquitetura, existem algumas métricas [Morgado, 2025]:

- Coesão - Nível de coerência na forma como os elementos do sistema estão agrupados.
- Acoplamento - Grau de interdependência entre as partes do sistema.
- Simplicidade - Nível de facilidade de compreensão/comunicação da arquitetura.
- Adaptabilidade - Nível de facilidade para alterar a arquitetura e incorporar novos requisitos.

No projeto, a coesão foi usada desde o começo para agrupar as partes responsáveis (coesão funcional). Por exemplo, na arquitetura detalhada, tudo o que diz respeito à apresentação está num *package* `apresentacao` e, tudo o que diz respeito ao acesso dos dados que estão numa base de dados encontra-se reunido no *package* `acesso_dados`.

No que toca ao acoplamento, manteve-se o menor que foi possível. Existem sempre relações que são imprescindíveis ao sistema, mas fez-se por manter apenas as relações necessárias. Estruturalmente não foi difícil evitar minimizar o acoplamento, devido à natureza simples do sistema.

Outros princípios utilizados foram a Fatorização, que ajuda a eliminar a redundância, e a Abstração que ajuda a realçar o essencial nas partes do sistema. Por exemplo, no projeto optou-se por criar uma única classe `funcionario` que serve para representar o Funcionário (Recepcionista), a Funcionária da Limpeza e o Proprietário do hotel.

Desta forma simplifica-se o diagrama e, conseqüentemente, o projeto e facilita-se a implementação, porque o sistema só lidará com uma classe e diferenciará os vários tipos pelo seu perfil.

O projeto baseou-se na arquitetura de três camadas, que define o aspecto geral da aplicação e a sua apresentação aos utilizadores.

O padrão escolhido para a camada da apresentação foi o padrão *Model-View-Controller* que tem o Modelo como parte responsável pela lógica da gestão do hotel e pela persistência dos dados; a Vista é responsável pela apresentação dos dados e o Controlador é responsável por receber informação do utilizador e coordenar a atualização do modelo e da vista.

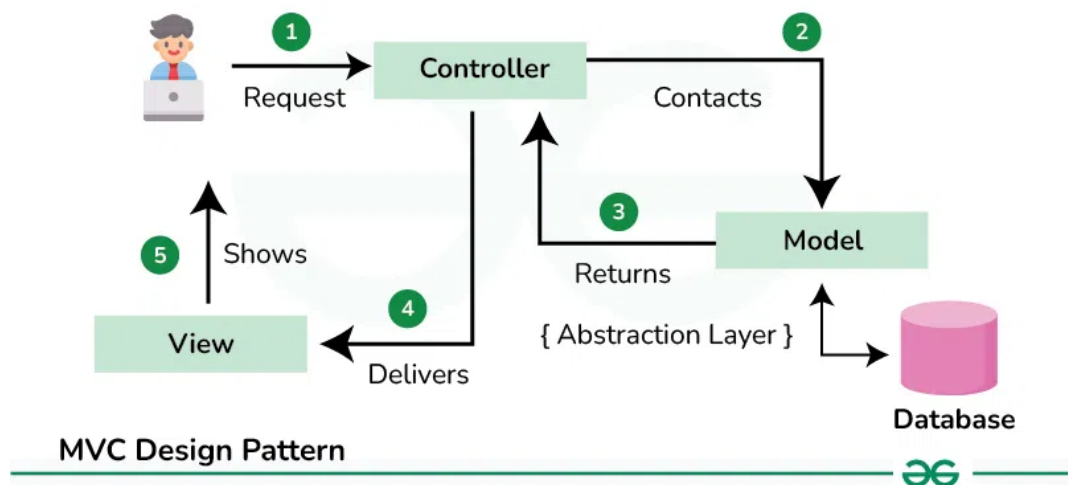


Figura 4.4: Diagrama que mostra o fluxo de funcionamento do padrão MVC [GeeksforGeeks, 2025]

Com esta informação passou-se à realização da arquitetura lógica.

Neste passo elaboraram-se os diagramas que servirão de base à implementação.

4.4 Arquitetura Detalhada

Por fim, chegou-se à última etapa antes da implementação.

Arquitetura de software é o nome que se dá à organização de um sistema, dos seus componentes e de como estes se relacionam entre si e com o ambiente.

Aqui já são tomadas decisões concretas sobre a estrutura dos dados, visibilidade e os métodos que lhes estarão associados e as linguagens que serão usadas para a implementação do sistema.

Voltaram-se a elaborar mais diagramas. Desta vez já com detalhes das linguagens e tecnologias a utilizar.

Criar-se os seguintes diagramas:

- Detalhe da Vista - Apresenta como as páginas HTML serão feitas e a que elementos recorrem para serem construídas. As páginas, além de HTML, também terão CSS para o estilo e JavaScript para funções que o cliente deve executar.
- Detalhe do acesso aos dados - Apresenta como os objetos DAO interagem com os repositórios através de MySQL.
- Detalhae da camda da API - Optou-se por usar uma API para chamar os serviços necessários e para existir uma melhor organização do projeto, criou-se um novo *package* `api` no *package* da Apresentação e do Domínio, sendo que o primeiro possui um Adaptador para o Cliente invocar os *endpoints* e o segundo tem uma classe com as rotas existentes e um adaptador para que o serviço correspondente à rota invocada seja chamado.
- Arquitetura de Teste - Para uma melhor implementação, optou-se por fazer uma arquitetura de teste, cujo *package* toma a vez da apresentação no diagrama e são criados repositórios de teste num simulador para aceder aos dados de teste.

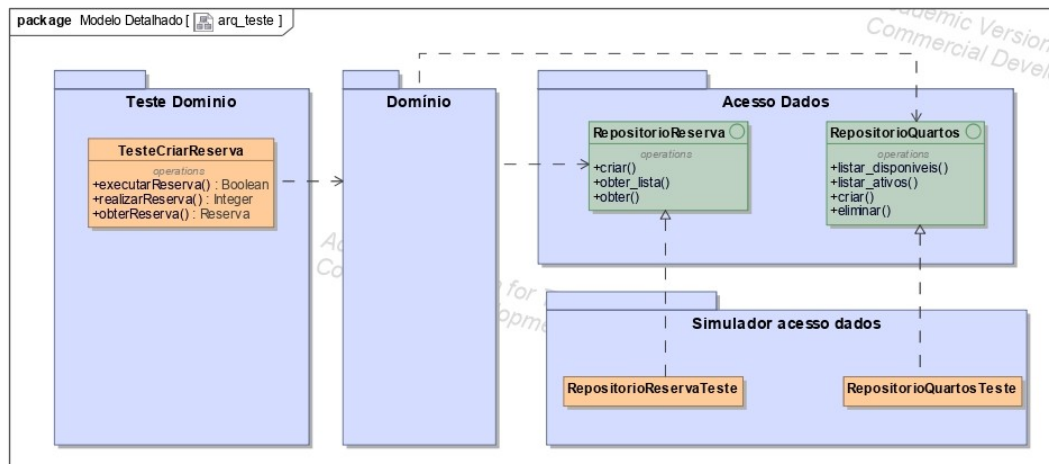


Figura 4.5: Arquitetura de Teste

Capítulo 5

Implementação do Protótipo da Solução

A implementação baseou-se em seguir os diagramas realizados nas etapas anteriores, principalmente nos da Arquitetura Detalhada. O caso escolhido para implementar foi o de **Criar Reserva**.

A linguagem escolhida foi Python para a arte da lógica de domínio, JavaScript para a apresentação e MySQL para o acesso aos dados.

5.1 Acesso aos Dados

Começou-se a implementação por escrever o código que define a estrutura da base de dados `hotel`. Embora não tivesse sido feito nenhum Modelo Entidade Associação próprio para o sistema, usou-se o Modelo de Domínio, que já dava uma boa noção de como ficaria a base de dados.

Corrigiu-se um erro aqui, pois a Relação entre **Reserva** e **Serviço** estava definida como 1:N, o que não faz muito sentido, pois todos os serviços deveriam estar disponíveis para todas as Reservas. Por este motivo, corrigiu-se esta relação para uma relação M:N.

Tendo em conta que o caso a ser implementado, só se criaram as tabelas das entidades que participam deste caso: `hospede`, `quarto`, `reserva`, `servico`, `tipo_quarto` e `reserva_servico`.

Em seguida populou-se a base de dados com alguns quartos, tipos de quartos e serviços, pois ao criar uma reserva pressupõe-se que estas entidades já estejam criadas e populadas.

O passo seguinte foi criar as classes que diziam respeito aos repositórios destas tabelas (Ver `es_50746_implementacao`).

5.2 Domínio

Para começar a parte de domínio, criaram-se as classes `reserva` e `estado_reserva`. Também se criou a classe do `servico_reservas`, mas apenas a parte estrutural. O mesmo foi feito com a entidade do hóspede, pois estes seriam as entidades que receberiam dados para a criação de uma reserva.

Também foram criadas uma pasta `tipos_dados` com a classe referente aos dados da reserva (`dados_reserva`) e outra aos do hóspede (`dados_hospede`).

O código que compõe o corpo das funções destas classes foi feito após a apresentação também estar em andamento, porque era mais fácil perceber se os dados teriam de ter mais algum tratamento do que aquele que já tinha sido planeado.

Para acabar esta camada de Domínio, também foi feito um novo package `api` com um `adaptador_servidor` e uma classe `rotas`. O Adaptador tem por base o `Flask` e serve para receber os pedidos que vêm do cliente com as rotas definidas, enquanto a classe que diz respeito às rotas tem o processo da rota propriamente dita e que invoca os serviços necessários.

5.3 Apresentação

Para começar a camada de apresentação, fez-se uma página HTML simples com CSS para o formulário que permite a criação de uma reserva com o registo de um hóspede (Ver `es_50746_implementacao`).

Tentou-se implementar a InfoView, mas a mecânica de *pop-up* não estava a funcionar e optou-se por cortá-la da versão final.

Além da página, foi criada uma pasta `api` que possui o `adaptador_cliente` que faz os pedidos à API do Sistema e aguarda a resposta. Este adaptador acabou por tomar o lugar do *controller* que estava presente na arquitetura. Esta mudança é fruto de algo que é referido no capítulo seguinte, que foi a dificuldade de perceber como as partes do MVC coexistem com uma API e, pela definição do *controller*, percebi que este tinha o papel que tinha atribuído ao Adaptador.

5.4 Testes

Os testes foram uma novidade para mim, porque nunca tinha trabalhado com um módulo específico para esta parte. Normalmente fazia testes intermédios com funções auxiliares feitas rapidamente quando era preciso, mas nunca nada pensado desde o início e que estivesse presente na arquitetura. Muitas dessas funções eram apagadas assim que o alvo do teste estava a funcionar como desejado.

Porém, neste projeto fiz quatro classes de teste: `criar_reserva`, `hospede`, `listar_consumos` e `listar_quartos_disp`.

As últimas duas eram um acesso direto à base de dados que serviu mais como um *spike*, pois nunca tinha mexido em MySQL com Python. As restantes classes chamavam os métodos de criar uma reserva e registar um hóspede e devolviam o sucesso, ou insucesso, da operação.

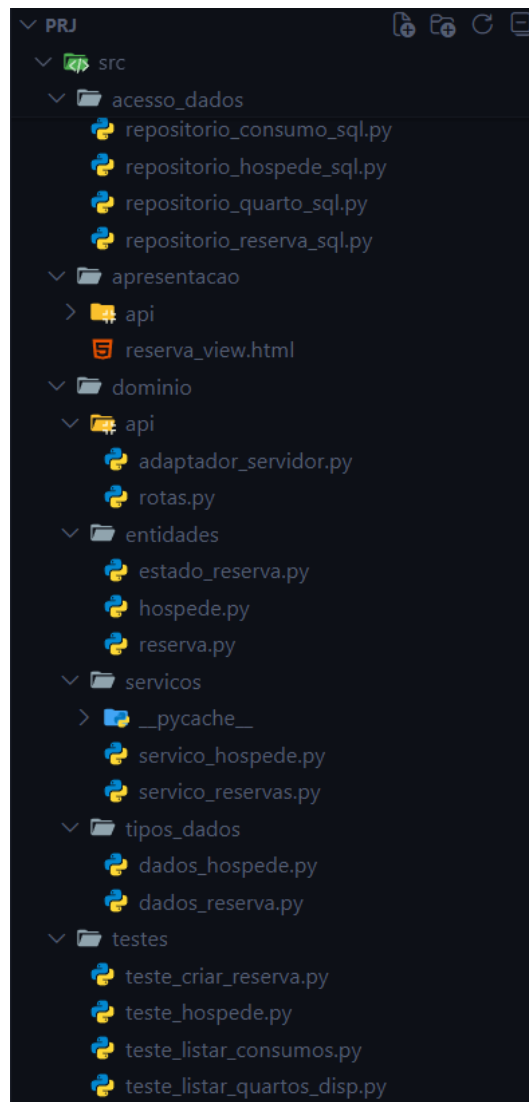


Figura 5.1: Estrutura da implementação

Capítulo 6

Revisão do Projeto Realizado

A maioria do projeto foi realizada nas aulas com a ajuda do professor.

A definição e elaboração da Visão de Solução foram fáceis de realizar e foram bastante úteis para começar a entender os contornos que o projeto tomaria no futuro.

A Análise de Requisitos também não apresentou problemas durante a sua realização. A descrição passo a passo de cada caso de utilização ajudou a aprofundar como seria o fluxo que o sistema teria e dos dados que seriam importantes recolher em cada fase dos diferentes casos.

O Modelo de Domínio também foi bastante intuitivo de fazer, porque é como se fosse um modelo de classes sem tanto detalhe, que são diagramas que já estamos habituados a fazer noutras unidades curriculares.

No que diz respeito à Modelação de Software, os conceitos que lhe dão base foram fáceis de perceber, como foi referido anteriormente (ver Capítulo 2). Contudo, existem conceitos que só na prática é que conseguimos perceber para que servem ou como aplicá-los.

Um bom exemplo, para a minha experiência durante o semestre, foram os Modelos de Dinâmica e os Modelos de Interação.

Nos modelos de dinâmica, por já ter trabalhado com máquinas de estado, consegui perceber os fundamentos base desta parte da matéria, mas em situações mais particulares tive dificuldade em entender o funcionamento de algumas especificações, como os Pseudo-Estado de Histórico Profundo. Não recorri a eles no meu trabalho. Contudo, achei-os ligeiramente abstratos ou que aumentariam a complexidade da leitura dos diagramas.

Porém, foi nos Diagramas de Sequência, dos Modelos de Interação, que

senti que o projeto me fugiu ligeiramente do controlo. Isto porque, embora percebesse o seu intuito e conceitos que lhe serviam de base, sentia que estava um pouco perdido na sua realização.

Penso que uma parte da minha dificuldade se deveu a não entender que cada Parte/Instância no Diagrama dizia respeito a uma parte do sistema já tendo em vista o padrão de arquitetura a ser utilizado. Depois, quando cheguei aos modelos da Arquitetura Lógica, tive alguma dificuldade em fazer esse "mapeamento".

O padrão de Arquitetura e a Arquitetura Lógica foram mais fáceis de entender do que implementar, principalmente o entendimento de como seria implementado o modelo MVC na camada de Apresentação quando existe uma API existente. Como nunca tinha sido apresentado a estes padrões de forma formal, tive dificuldade em perceber a função de cada componente. Mas estas dificuldades foram apaziguadas e contornadas com a ajuda do professor.

A Arquitetura Detalhada não apresentou tantas dificuldades como a etapa anterior. A única questão que se levantou foi como representar a criação e utilização de uma API, mas o professor conseguiu esclarecer esta dúvida.

Por último, na fase de implementação e testes, a única dificuldade foi o tempo disponível para realizá-las, pois calham numa altura crítica do semestre, mas conseguiu-se fazer a implementação do caso de utilização escolhido seguindo a ideia original. Existiram algumas diferenças, embora pequenas, entre alguns diagramas e a sua implementação. Isto porque, como referido anteriormente, nalgumas fases do projeto nas quais já é suposto saber alguns atributos que serão passados a funções e como lidar com determinados erros, é difícil fazê-lo quando ainda não se tem muita experiência. Contudo, acredito que este seja um aspeto que tenho a melhorar para futuros trabalhos.

Acredito que, se tivesse tido mais tempo na implementação, tentaria implementar mais alguns casos e sem pressuposições, como ocorreu no projeto, para conseguir focar-me apenas na parte importante do caso de utilização.

Na fase de testes, foi interessante experimentar ter módulos próprios para esta finalidade, pois não tinha trabalhado desta forma antes.

Como nota adicional, gostaria de dizer que, embora na sua maioria o tempo seja bem distribuído pelos diferentes temas pelo professor, acredito que existem pontos como os modelos de dinâmica que, na minha opinião, entram

demasiado em pormenor e acabam por baralhar o aluno. Se nalguns temas o tempo despendido fosse menor, talvez ainda houvesse tempo de realizar uma parte da implementação na sala de aula e, deste modo, aprimorar mais este aspeto do trabalho com a calma e atençãoq que lhe é necessária.

Capítulo 7

Conclusão

Com este trabalho ficou nítida a importância de organizar e planejar um projeto desde o seu começo. Já tinha ganho essa noção com o projeto final da licenciatura; contudo, nesta unidade curricular, consegui perceber várias nuances desta matéria que antes não conhecia.

Uma boa parte dos temas já me era familiar, mais concretamente, até o tópico da arquitetura detalhada, por serem temas compartilhados e lecionados noutras cadeiras. Porém, conheci vários pormenores da linguagem UML, principalmente nos modelos de interação e dinâmica que me eram totalmente desconhecidos, e percebi o seu valor em projetos complexos e que requerem um alto detalhe.

Sem dúvida que o conhecimento neste campo é importante, ainda mais nos dias que correm.

Bibliografia

[GeeksforGeeks, 2025] GeeksforGeeks (Acedido: 2025). Mvc design pattern. Disponível em: <https://www.geeksforgeeks.org/system-design/mvc-design-pattern/>. Acesso em: 2025.

[Morgado, 2025] Morgado, L. (2025). Documentação de apoio à unidade curricular engenharia de software. Disponibilizado via Moodle. Acedido: 2025.